



Bridge

Padrões de Projeto Estrutural II

Prof. Me Jefferson Passerini



Bridge design pattern

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

O padrão **Bridge** desacopla uma abstração de sua implementação, ou seja, permite dividir uma classe grande ou um conjunto de classes diretamente ligadas em duas hierarquias separadas, deste modo as duas hierarquias podem variar de forma independente.

Aplicabilidade

- Quando se deseja evitar uma ligação permanente entre uma abstração e sua implementação. Pode ser o caso, por exemplo, quando a implementação deve ser selecionada ou alternada no tempo de execução.
- Quando tanto as abstrações quanto suas implementações devem ser extensíveis por subclassificação. Nesse caso, o padrão *Bridge* permite combinar as diferentes abstrações e implementações e estendê-las independentemente.
- Quando mudanças na implementação de uma abstração não devem causar impacto nos clientes (o cliente não pode ser recompilado).
- Quando existe uma proliferação de classes. Essa hierarquia de classes indica a necessidade de dividir um objeto em duas partes.
- Quando se deseja compartilhar uma implementação entre vários objetos, e esse fato deve estar oculto no cliente.

Componentes

- **Abstração:** Define a interface da Abstração e mantém uma referência a um objeto do tipo Implementador.
- **RedefinicaoDaAbstracao:** Estende a interface definida por Abstração. É opcional.

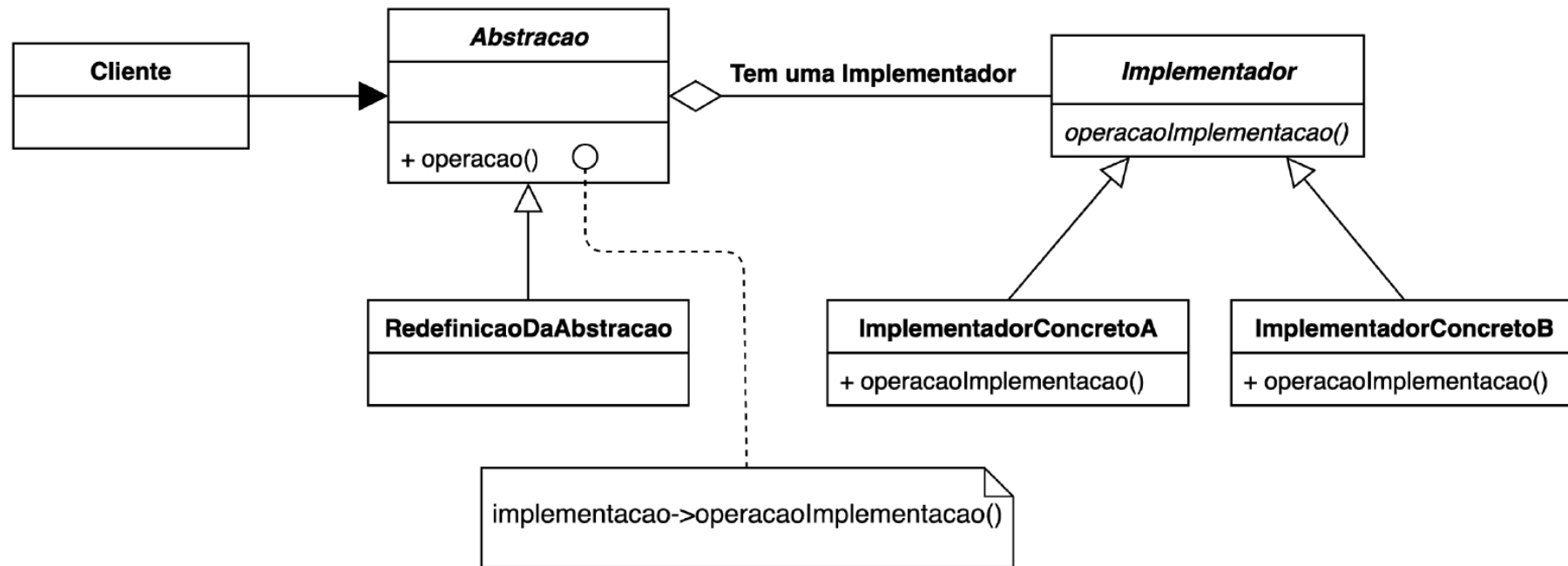


Diagrama de Classes

Componentes

- **Implementador:** Define a interface para as classes de Implementador. Essa interface não precisa ser igual a interface Abstracao, na verdade, as duas interfaces podem ser bem diferentes. Normalmente, a interface do Implementador fornece apenas operações primitivas e a Abstracao define operações de nível superior com base nessas primitivas.

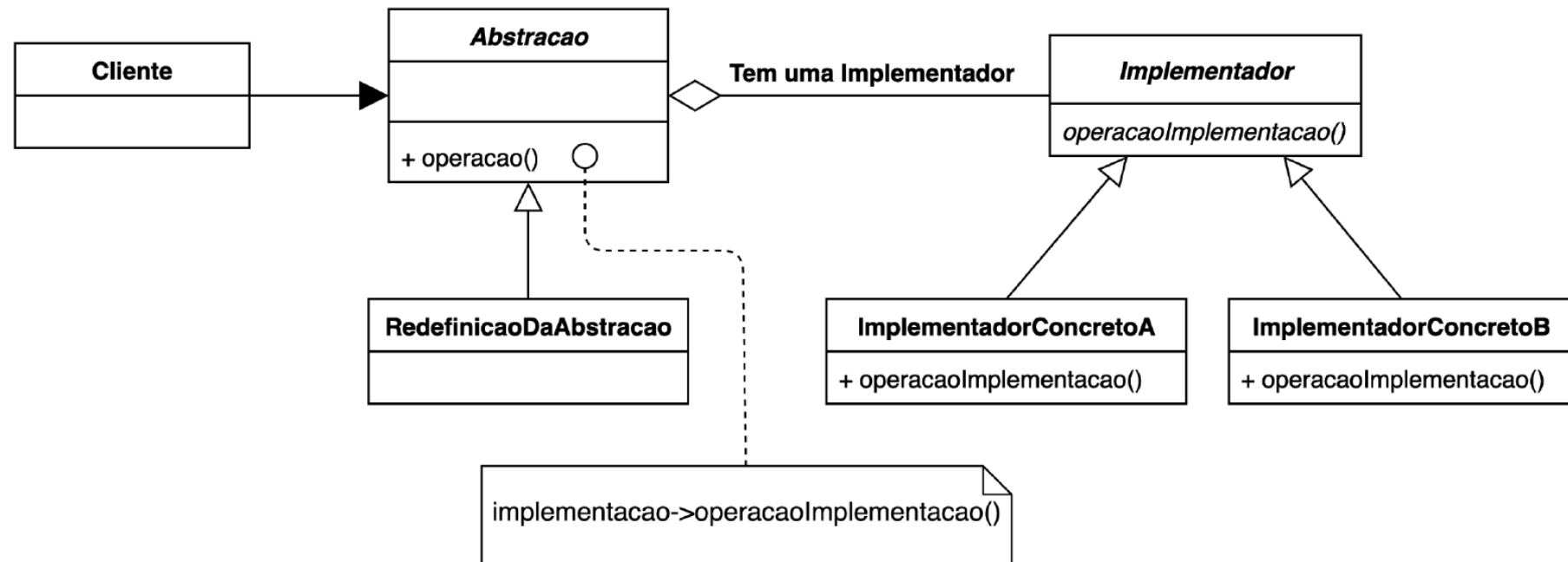


Diagrama de Classes

Componentes

- **ImplementadorConcreto:** Implementa a interface do Implementador e define sua implementação concreta.

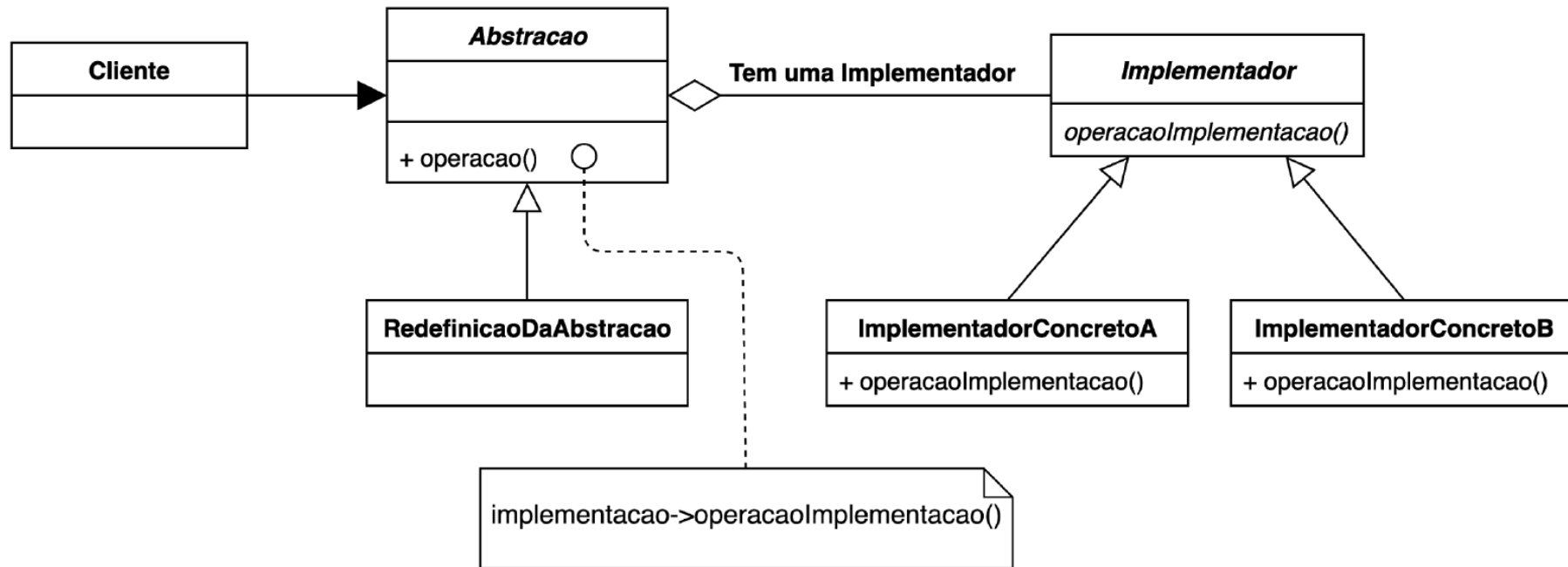


Diagrama de Classes

Motivação (Porque Utilizar?)

- Quando uma abstração pode ter uma entre várias implementações possíveis, a maneira usual de implementar isso é usando herança.
- Uma classe abstrata define a interface para a abstração, e subclasses concretas que a implementam de maneiras diferentes, mas, essa abordagem nem sempre é flexível o suficiente.
- A **herança vincula permanentemente uma implementação à abstração**, o que torna difícil modificar, estender e reutilizar abstrações e implementações de forma independente.
- *Imagine um módulo de troca de mensagens entre administrador e usuários de um sistema, suponha que a mensagem enviada pelo administrador é diferente da enviada pelo usuário.*
- *Inicialmente precisamos apenas da mensagem e que tanto administrador quanto usuário sejam capazes de enviá-las.*

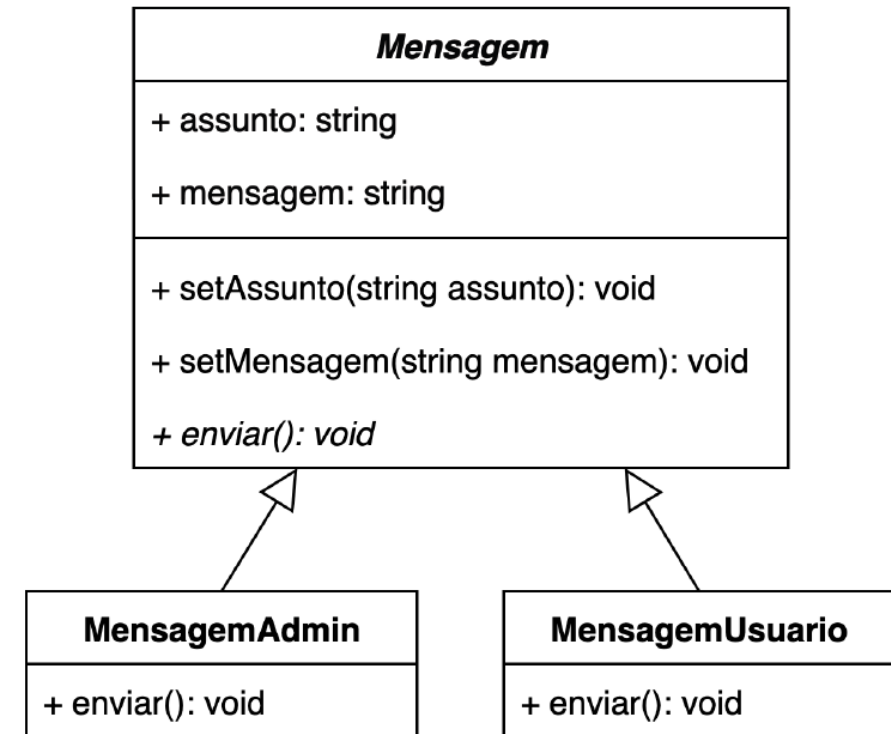


Diagrama de classes de envio de mensagens

Motivação (Porque Utilizar?)

- Temos duas implementações possíveis para **Mensagem**, repare no diagrama, que apenas criando as subclasses temos uma solução satisfatória.
- **MensagemAdmin** envia mensagens do administrador para os usuários e **MensagemUsuario** envia mensagens dos usuários para o administrador.

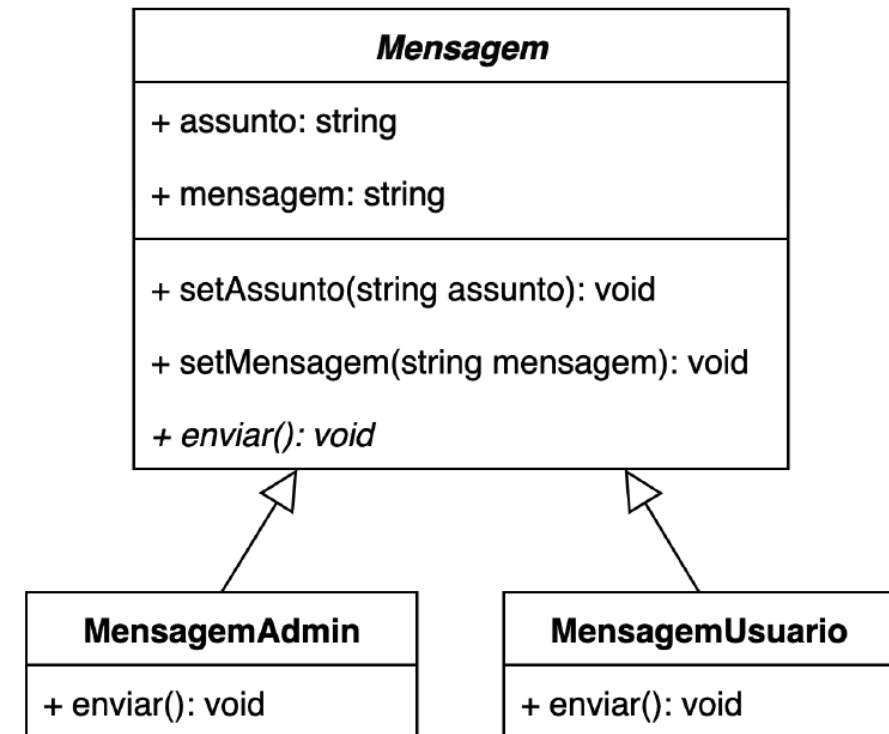


Diagrama de classes de envio de mensagens

Motivação (Porque Utilizar?)

- Agora considere que a mensagem pode ser enviada por **e-mail** ou por **SMS**. Vejamos como ficaria nosso diagrama de classes seguindo a mesma abordagem de herança.
- A quantidade de classes no diagrama dobrou, tal problema ocorre porque estamos tentando estender as classes de **Mensagem** em duas dimensões diferentes:
 - Por tipo de usuário.
 - Por canal de envio da mensagem.
- Este é um **problema muito comum com herança** de classes. Imagine que a mensagem também precisa ser **enviada por WhatsApp**, seria necessário criar mais duas classes **MensagemAdminWhats** e **MensagemUsuarioWhats**.
- A cada novo canal de envio de mensagem ou novo tipo de usuário, a quantidade de subclasses iria crescer.

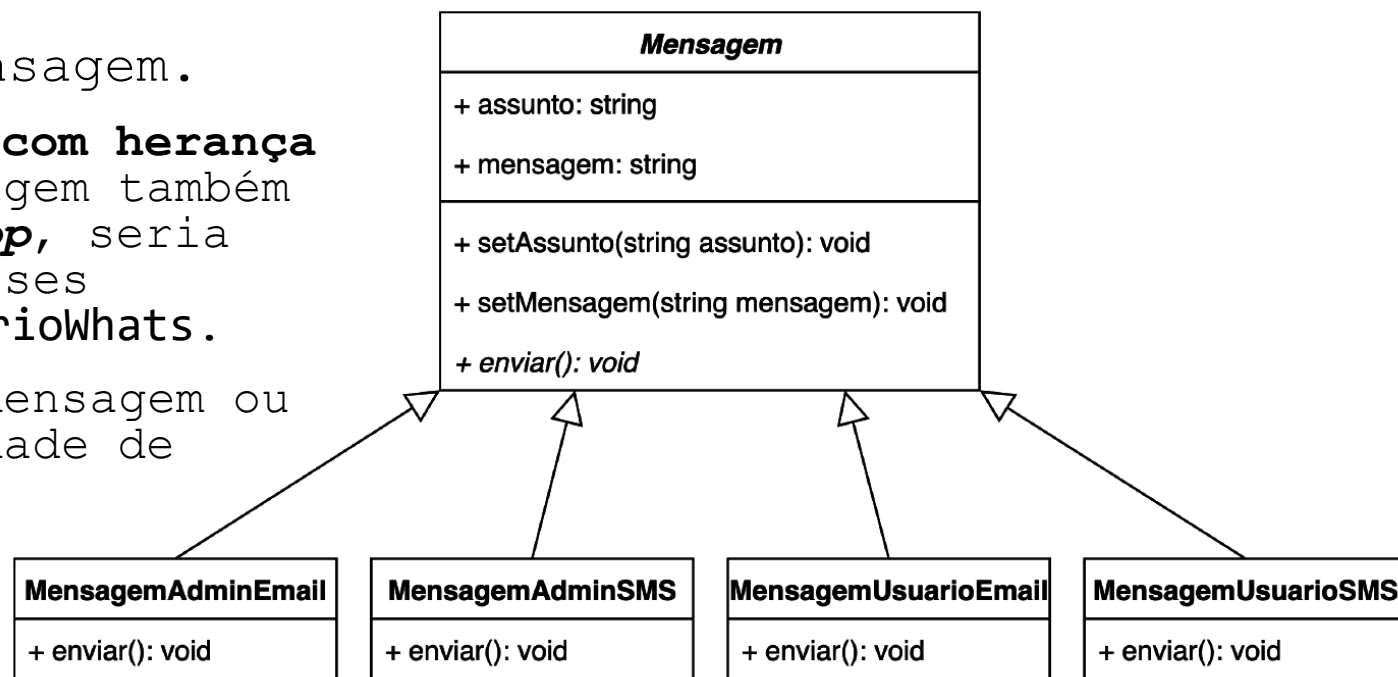


Diagrama de classes de envio de mensagens por email e SMS

- Existe um princípio de OO que diz “**Priorizar a composição ao invés da herança**”.
- O padrão *Bridge* tenta resolver o problema de subclassificação seguindo este princípio.
- Isso significa que uma das dimensões deve ser extraída da hierarquia de **Mensagem** e criar uma hierarquia de classes separada. Deste modo, as classes originais irão referenciar um objeto da nova hierarquia, ao invés de ter todos os seus estados e comportamentos dentro de uma única classe.
- Vamos extrair o canal de envio da hierarquia de **Mensagem**. Assim ela será subclassificada apenas em termo de tipo de usuário e irá referenciar a hierarquia de canal de envio.

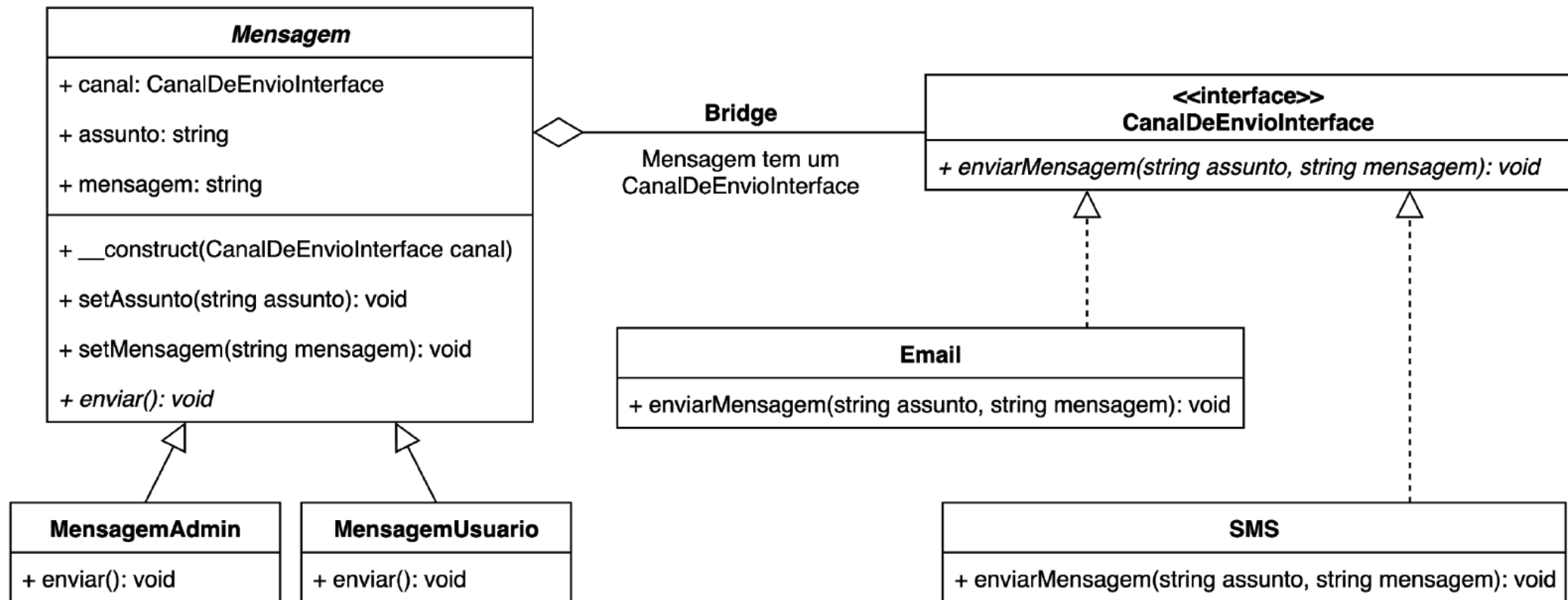


Diagrama de classes de envio de mensagens por email e SMS utilizando o padrão Bridge

- Agora existem hierarquias de classes separadas, uma para o tipo de usuário e outra para o canal de envio.
- Com a nova configuração a **Mensagem** pode delegar qualquer trabalho referente ao canal de envio para o objeto ligado a **CanalDeEnvioInterface**. Essa referência vai agir como uma ponte (em inglês *bridge*) entre as classes **Mensagem** e **CanalDeEnvioInterface**.
- De agora em diante, para adicionar novos canais de envio ou novos usuários não será necessário mudar a hierarquia de **Mensagem** e vice-versa

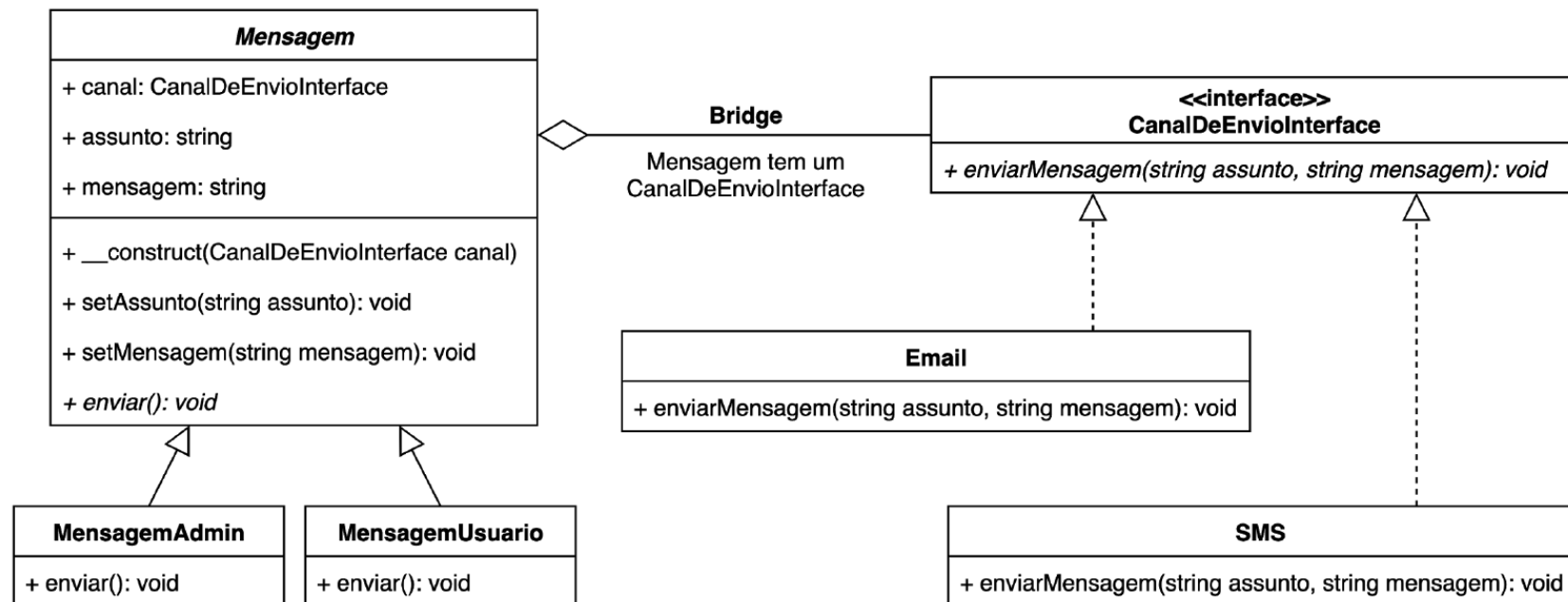


Diagrama de classes de envio de mensagens por email e SMS utilizando o padrão Bridge



Bridge Implementação em C#

Padrões de Projeto Estrutural II

Prof. Me Jefferson Passerini



Bridge design pattern

Loosen the coupling between the bridge deck and the bridge tower.
The bridge deck is a separate object that can be used in other contexts.
The bridge tower is a separate object that can be used in other contexts.

- Agora crie seu projeto em C# console e prepare a seguinte estrutura de pastas. Para armazenar nossa estrutura proposta.

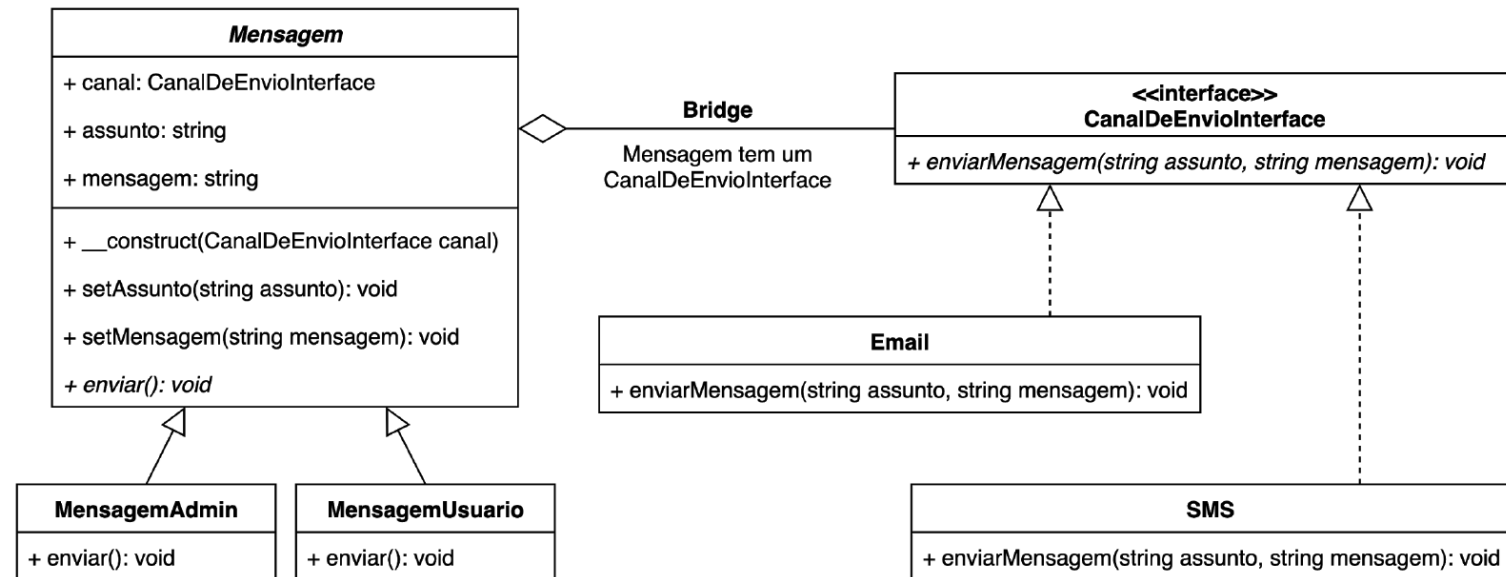
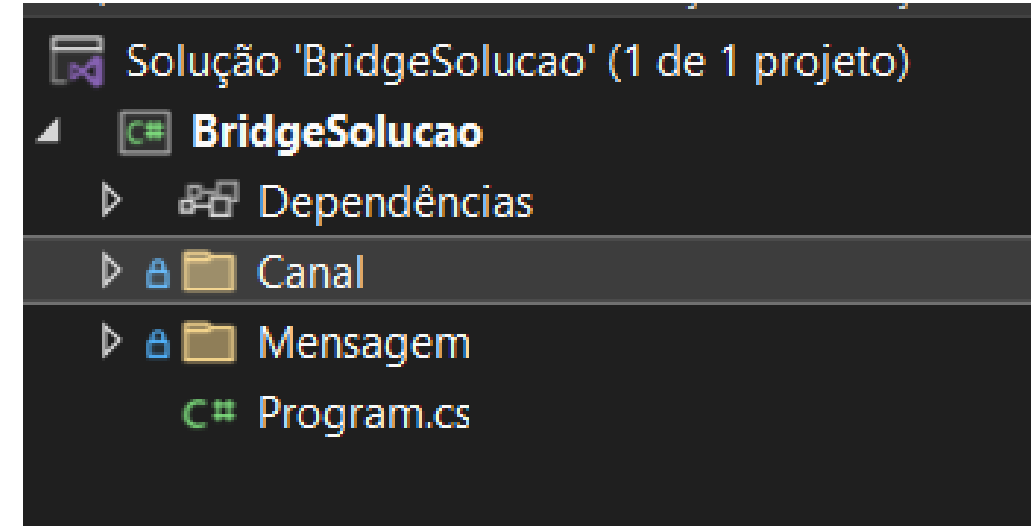


Diagrama de classes de envio de mensagens por email e SMS utilizando o padrão Bridge

- Crie um arquivo para a interface CanalDeEnvio e declare a assinatura do método que deverá ser implementado pelas classes concretas.

```
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Text;
5  using System.Threading.Tasks;
6
7  namespace BridgeSolucao.Canal
8  {
9      0 referências
10     public interface CanalDeEnvio
11     {
12         0 referências
13         public void enviarMensagem(String assunto, String mensagem);
14     }
15 }
```

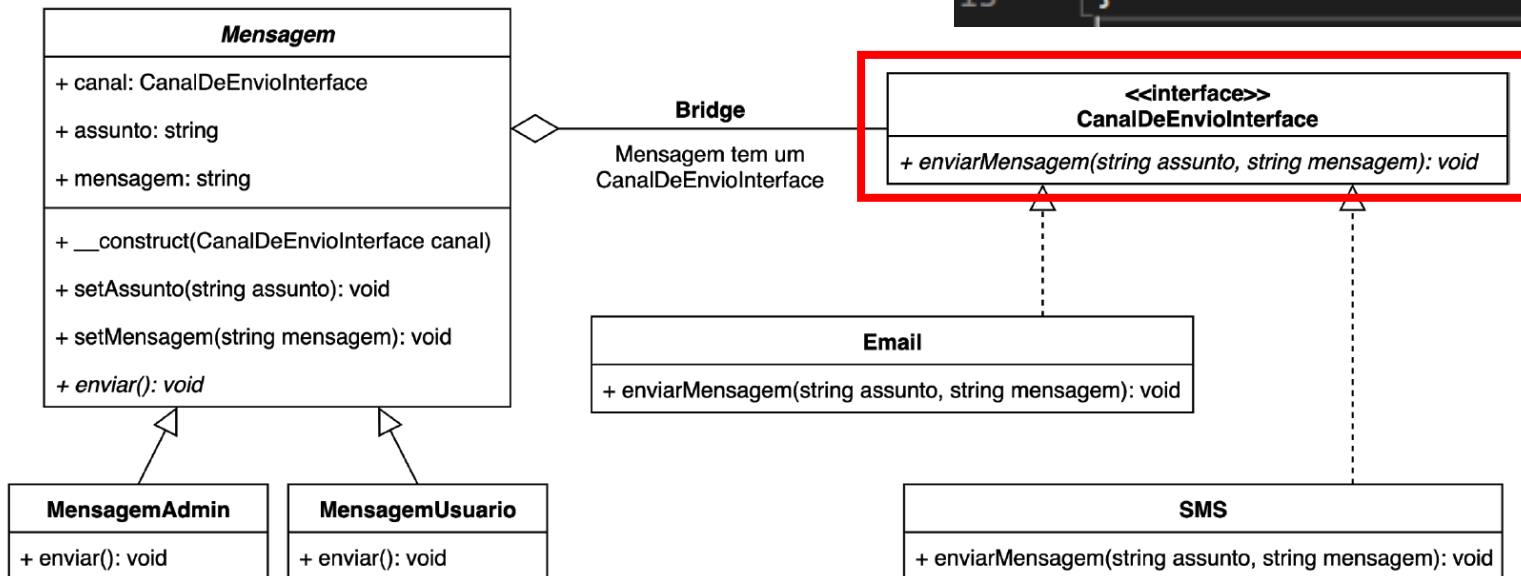
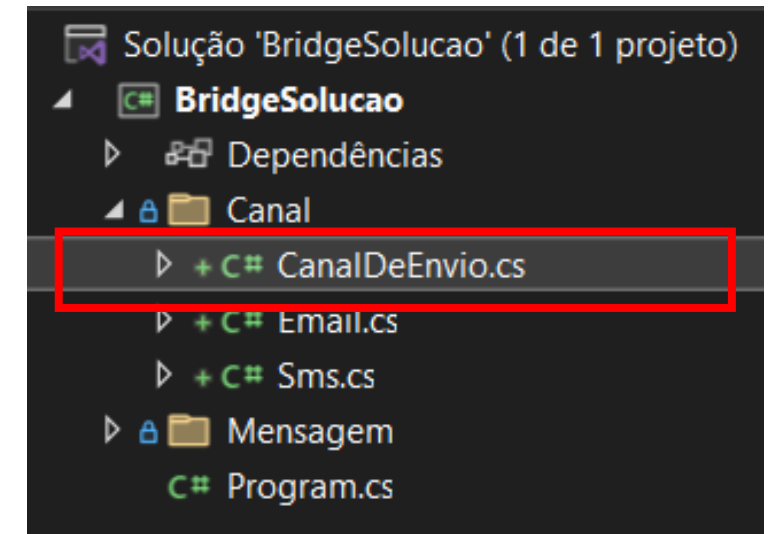


Diagrama de classes de envio de mensagens por email e SMS utilizando o padrão Bridge



- Crie um arquivo para a classe Email que irá implementar o método enviarMensagem().

```

1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Text;
5  using System.Threading.Tasks;
6
7  namespace BridgeSolucao.Canal
8  {
9      1 referência
10     public class Email : CanalDeEnvio
11     {
12         2 referências
13         public void enviarMensagem(string assunto, string mensagem)
14         {
15             String saida = $"Email enviado: {assunto} - Mensagem: {mensagem}";
16             Console.WriteLine(saida);
17         }
18     }
19 }

```

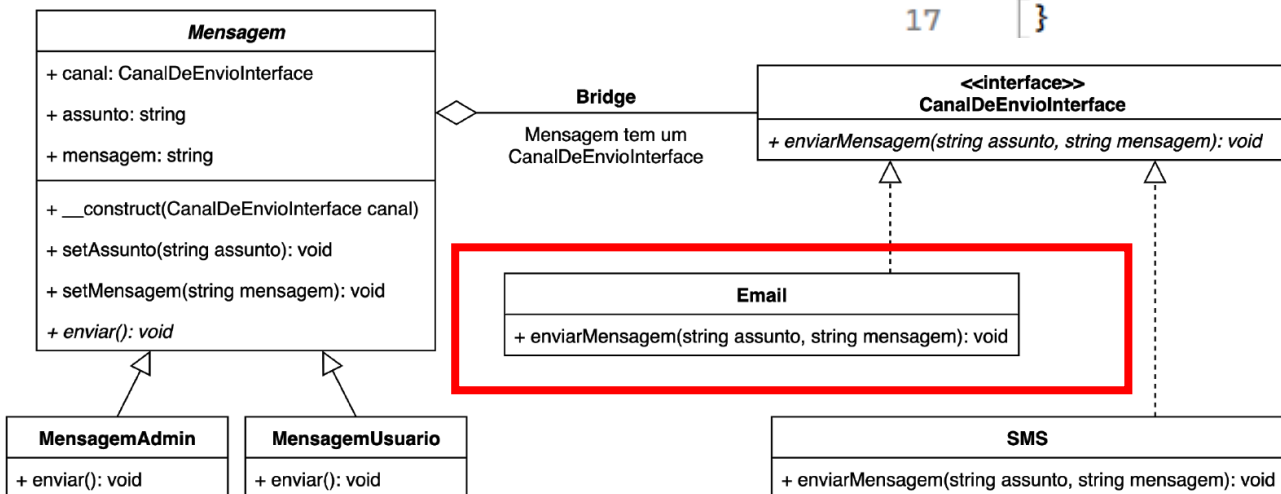
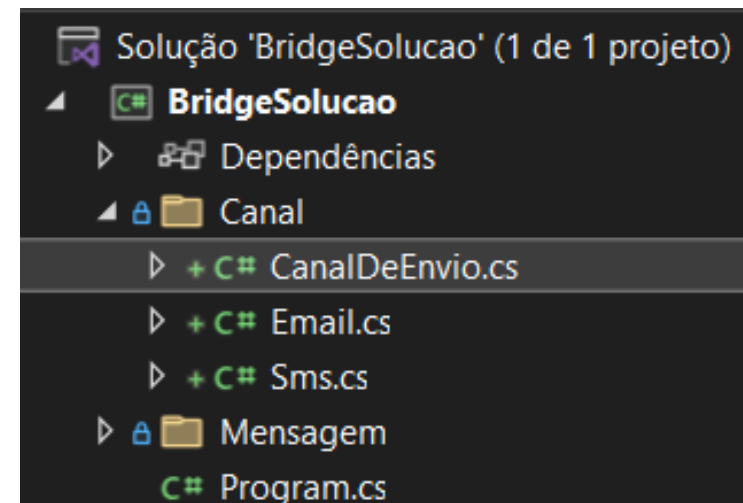


Diagrama de classes de envio de mensagens por email e SMS utilizando o padrão Bridge



- Crie um arquivo para a classe Sms que irá implementar o método enviarMensagem().

```
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Text;
5  using System.Threading.Tasks;
6
7  namespace BridgeSolucao.Canal
8  {
9      0 referências
10     public class Sms : CanalDeEnvio
11     {
12         2 referências
13         public void enviarMensagem(string assunto, string mensagem)
14         {
15             String saida = $"SMS enviado: {assunto} - Mensagem: {mensagem}";
16             Console.WriteLine(saida);
17         }
18     }
19 }
```

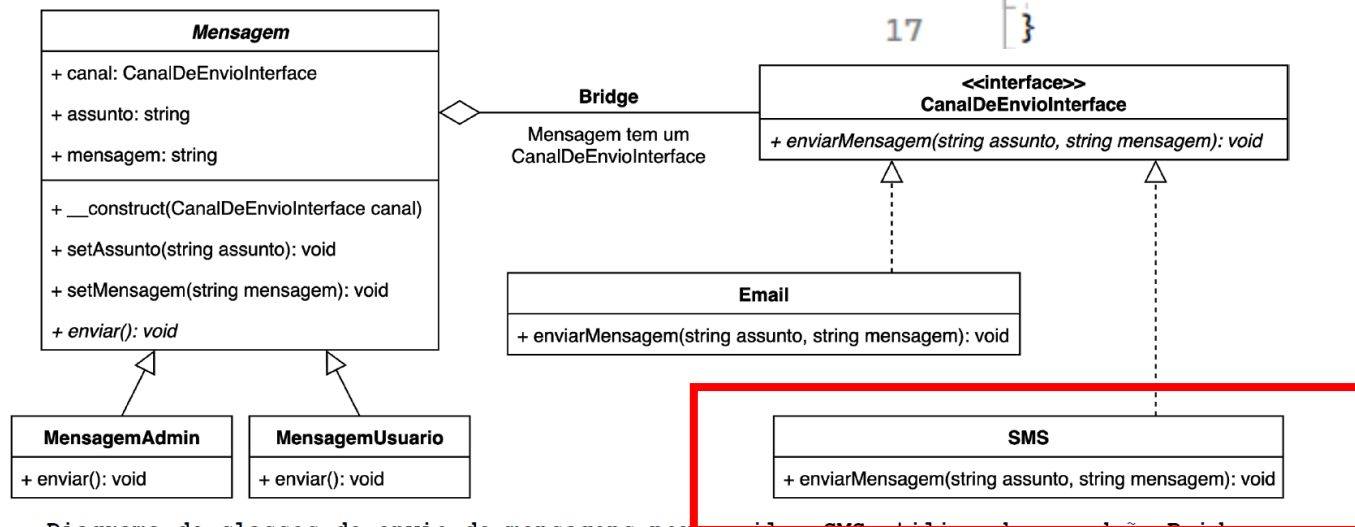
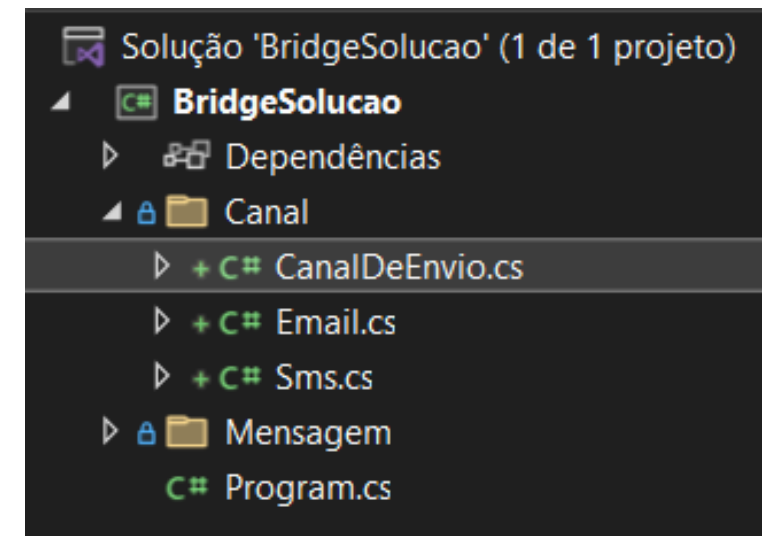


Diagrama de classes de envio de mensagens por Email e SMS utilizando o padrão Bridge



Padrões de Projetos Estrutural – Bridge

- Crie a classe abstrata Mensagem.

```
1 using BridgeSolucao.Canal;
2 using System;
3 using System.Collections.Generic;
4 using System.Linq;
5 using System.Text;
6 using System.Threading.Tasks;
7
8 namespace BridgeSolucao.Mensagem
9 {
10     1 referência
11     public abstract class Mensagem
12     {
13         1 referência
14         public CanalDeEnvio canal { get; set; }
15         2 referências
16         public String assunto { get; set; }
17         2 referências
18         public String mensagem { get; set; }
19
20         0 referências
21         public Mensagem(CanalDeEnvio canal) {
22             this.canal = canal;
23             this.assunto = "";
24             this.mensagem = "";
25         }
26
27         public void setAssunto(String assunto)
28         {
29             this.assunto = assunto;
30         }
31
32         0 referências
33         public void setMensagem(String mensagem)
34         {
35             this.mensagem = mensagem;
36         }
37
38         0 referências
39         public abstract void enviarMensagem();
40     }
41 }
```

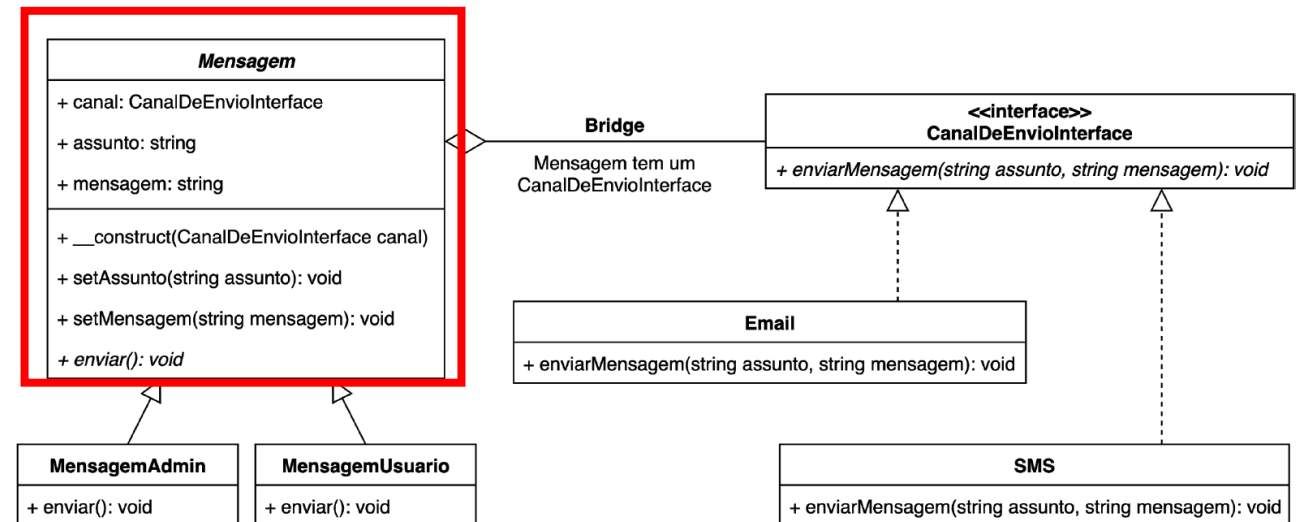
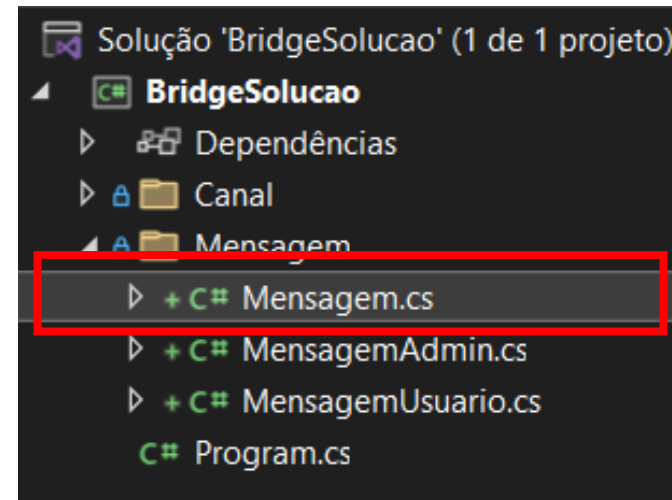
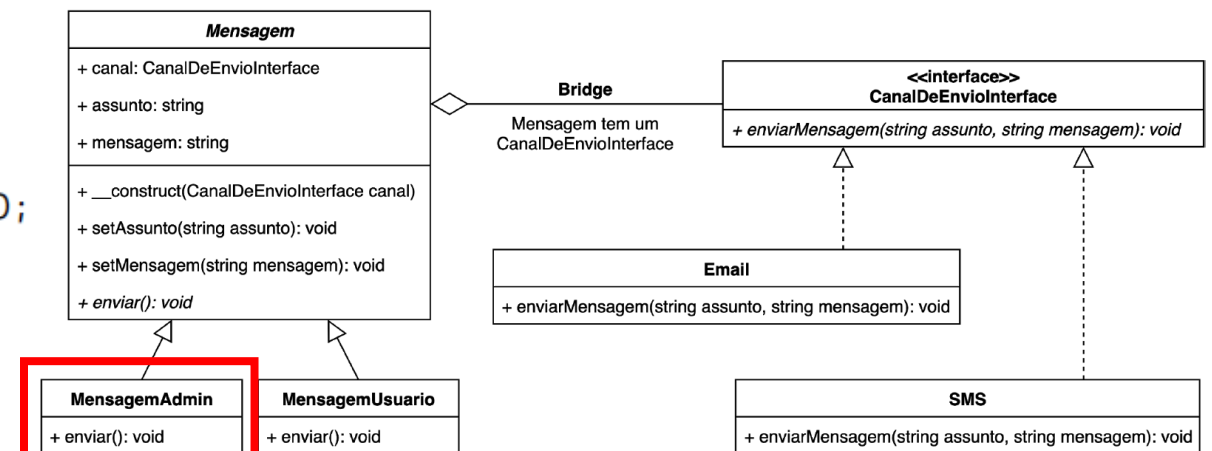
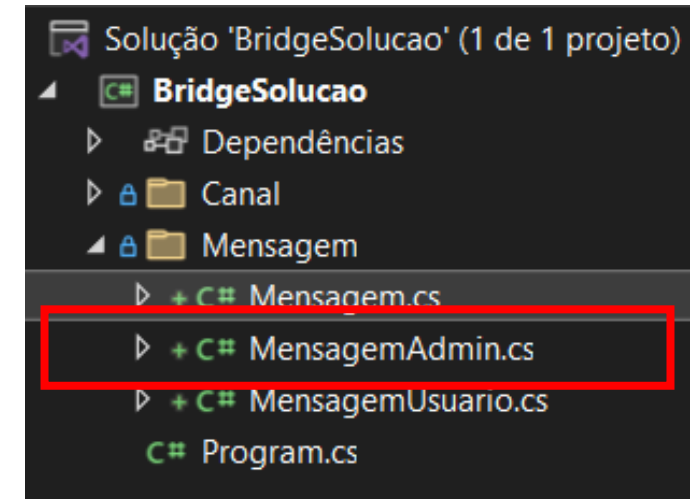


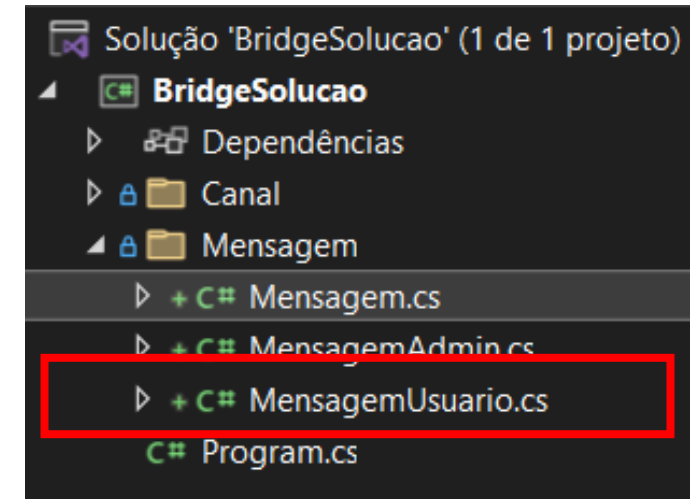
Diagrama de classes de envio de mensagens por email e SMS utilizando o padrão Bridge

- Crie a classe MensagemAdmin

```
1 using BridgeSolucao.Canal;
2 using System;
3 using System.Collections.Generic;
4 using System.Linq;
5 using System.Text;
6 using System.Threading.Tasks;
7 using System.Transactions;
8
9 namespace BridgeSolucao.Mensagem
10 {
11     2 referências
12     public class MensagemAdmin : Mensagem
13     {
14         1 referência
15         public MensagemAdmin(CanalDeEnvio canal) : base(canal)
16         {
17         }
18
19         2 referências
20         public override void enviarMensagem()
21         {
22             Console.WriteLine("Msg Enviada pelo Administrador");
23             base.canal.enviarMensagem(base.assunto, base.mensagem);
24         }
25     }
26 }
```



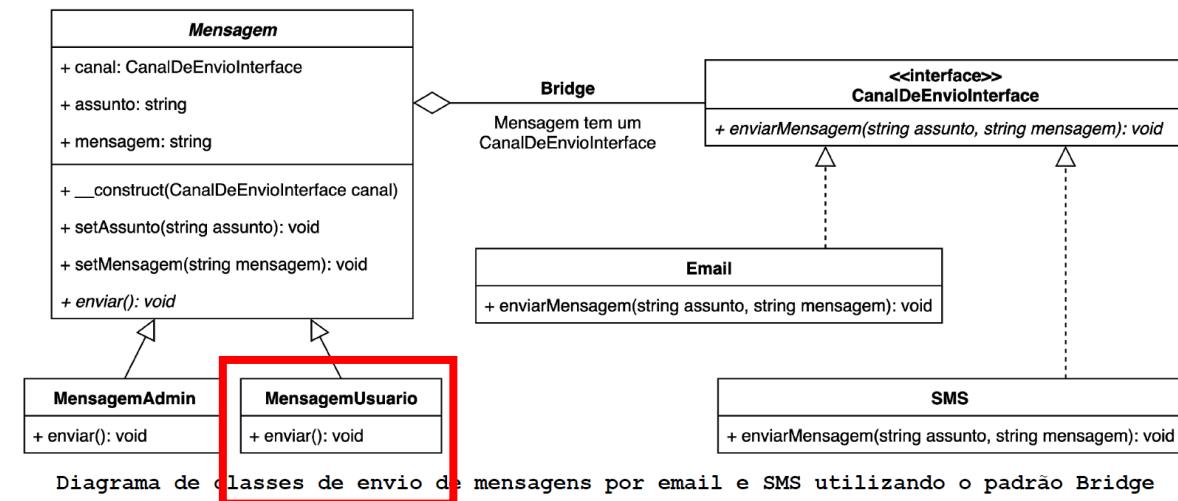
- Crie a classe MensagemUsuario.



```

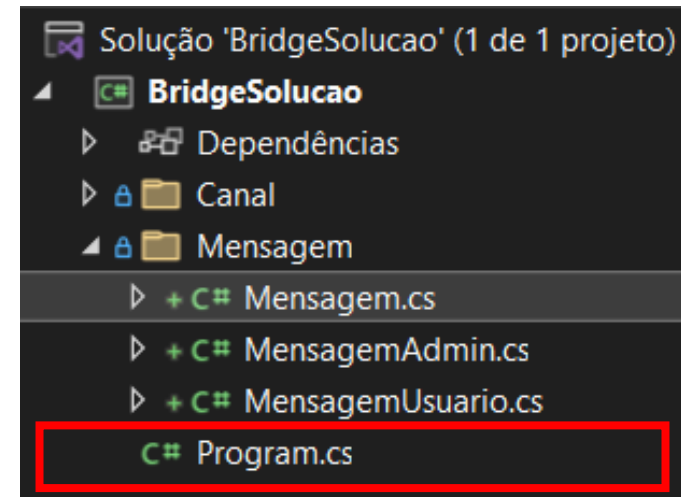
1  using BridgeSolucao.Canal;
2  using System;
3  using System.Collections.Generic;
4  using System.Linq;
5  using System.Text;
6  using System.Threading.Tasks;
7
8  namespace BridgeSolucao.Mensagem
9  {
10     3 referências
11     public class MensagemUsuario : Mensagem
12     {
13         2 referências
14         public MensagemUsuario(CanalDeEnvio canal) : base(canal)
15         {
16         }
17
18         5 referências
19         public override void enviarMensagem()
20         {
21             Console.WriteLine("Msg Enviada pelo usuario");
22             base.canal.enviarMensagem(base.assunto, base.mensagem);
23         }
24     }

```



```
1  using BridgeSolucao.Canal;
2  using BridgeSolucao.Mensagem;
3
4  CanalDeEnvio canalEmail = new Email();
5
6  Mensagem msgAdmin = new MensagemAdmin(canalEmail);
7  msgAdmin.setAssunto("Primeira Mensagem");
8  msgAdmin.setMensagem("Olá Usuário!");
9  msgAdmin.enviarMensagem();
10
11 Mensagem msgUsuario = new MensagemUsuario(canalEmail);
12 msgUsuario.setAssunto("Primeira Mensagem");
13 msgUsuario.setMensagem("Olá Usuário!");
14 msgUsuario.enviarMensagem();
15
16 CanalDeEnvio canalSms = new Sms();
17
18 Mensagem msgAdmin2 = new MensagemAdmin(canalSms);
19 msgAdmin.setAssunto("Primeira Mensagem");
20 msgAdmin.setMensagem("Olá Usuário!");
21 msgAdmin.enviarMensagem();
22
23 Mensagem msgUsuario2 = new MensagemUsuario(canalSms);
24 msgUsuario.setAssunto("Primeira Mensagem");
25 msgUsuario.setMensagem("Olá Usuário!");
26 msgUsuario.enviarMensagem();
```

- Classe Program para testarmos o modelo.



```
Msg Enviada pelo Administrador
Email enviado: Primeira Mensagem - Mensagem: Olá Usuário!
Msg Enviada pelo usuario
Email enviado: Primeira Mensagem - Mensagem: Olá Usuário!
Msg Enviada pelo Administrador
Email enviado: Primeira Mensagem - Mensagem: Olá Usuário!
Msg Enviada pelo usuario
Email enviado: Primeira Mensagem - Mensagem: Olá Usuário!
```



Bridge Implementação em Java

Padrões de Projeto Estrutural II

Prof. Me Jefferson Passerini



Bridge design pattern

Bridge design pattern

Լուսին Իսթանբուլից հեռացավ և արժանացավ ժողովուրդի օրհնանքի,
 ժողովուրդի օրհնանքի և արժանացավ ազատության և անկախության։
 Նոր Բիթլիսի ժողովը և Երևանի ժողովը և Երևանի ժողովը։

- Desacopla a interface da implementação. Uma implementação não fica vinculada permanentemente a uma interface. A implementação de uma abstração pode ser configurada em tempo de execução. É até possível para um objeto alterar sua implementação em tempo de execução.
- O desacoplamento de **Abstracao** e **Implementador** também elimina dependências de tempo de compilação na implementação. Alterar uma classe de implementação não requer recompilação da classe **Abstracao** e seus clientes.
- Além disso, tal desacoplamento incentiva o uso de camadas que podem levar a um sistema melhor estruturado. A parte de alto nível de um sistema precisa apenas conhecer **Abstracao** e **Implementador**.

- Extensibilidade aprimorada. É possível estender as hierarquias de **Abstracao** e **Implementador** de forma independente.
- Ocultação de detalhes de implementação dos clientes. é possível proteger os clientes de detalhes de implementação, tais como o compartilhamento de objetos do **Implementador**.